Name: Pradeepa. S Register Number: 250701531

```
import numpy as np
import pandas as pd

df = pd.read_csv('pre-process_datasample.csv')

df['Country'].fillna(df['Country'].mode(), inplace=True)
df['Age'].fillna(df['Age'].median(), inplace=True)
df['Salary'].fillna(round(df['Salary'].mean()), inplace=True)

dummy_countries = pd.get_dummies(df['Country'])
updated_dataset = pd.concat([dummy_countries, df.iloc[:, 1:4]],
axis=1)
updated_dataset['Purchased'].replace(['No', 'Yes'], inplace=True)

print(updated_dataset)
```

```
   France  Germany  Spain   Age   Salary Purchased
0    True    False  False  44.0  72000.0        No
1   False    False   True  27.0  48000.0        No
2   False     True  False  30.0  54000.0        No
3   False    False   True  38.0  61000.0        No
4   False     True  False  40.0  63778.0        No
5    True    False  False  35.0  58000.0        No
6   False    False   True  38.0  52000.0        No
7    True    False  False  48.0  79000.0        No
8   False     True  False  50.0  83000.0        No
9    True    False  False  37.0  67000.0        No

C:\Users\swath\AppData\Local\Temp\ipykernel_22860\328809920.py:6:
FutureWarning: A value is trying to be set on a copy of a DataFrame or
Series through chained assignment using an inplace method.
The behavior will change in pandas 3.0. This inplace method will never
work because the intermediate object on which we are setting values
always behaves as a copy.

For example, when doing 'df[col].method(value, inplace=True)', try
using 'df.method({col: value}, inplace=True)' or df[col] =
df[col].method(value) instead, to perform the operation inplace on the
original object.


  df['Country'].fillna(df['Country'].mode(), inplace=True)
C:\Users\swath\AppData\Local\Temp\ipykernel_22860\328809920.py:7:
FutureWarning: A value is trying to be set on a copy of a DataFrame or
Series through chained assignment using an inplace method.
The behavior will change in pandas 3.0. This inplace method will never
work because the intermediate object on which we are setting values
```

```
always behaves as a copy.

For example, when doing 'df[col].method(value, inplace=True)', try
using 'df.method({col: value}, inplace=True)' or df[col] =
df[col].method(value) instead, to perform the operation inplace on the
original object.


  df['Age'].fillna(df['Age'].median(), inplace=True)
C:\Users\swath\AppData\Local\Temp\ipykernel_22860\328809920.py:8:
FutureWarning: A value is trying to be set on a copy of a DataFrame or
Series through chained assignment using an inplace method.
The behavior will change in pandas 3.0. This inplace method will never
work because the intermediate object on which we are setting values
always behaves as a copy.

For example, when doing 'df[col].method(value, inplace=True)', try
using 'df.method({col: value}, inplace=True)' or df[col] =
df[col].method(value) instead, to perform the operation inplace on the
original object.


  df['Salary'].fillna(round(df['Salary'].mean()), inplace=True)
C:\Users\swath\AppData\Local\Temp\ipykernel_22860\328809920.py:12:
FutureWarning: Series.replace without 'value' and with non-dict-like
'to_replace' is deprecated and will raise in a future version.
Explicitly specify the new values instead.
  updated_dataset['Purchased'].replace(['No', 'Yes'], inplace=True)
C:\Users\swath\AppData\Local\Temp\ipykernel_22860\328809920.py:12:
FutureWarning: A value is trying to be set on a copy of a DataFrame or
Series through chained assignment using an inplace method.
The behavior will change in pandas 3.0. This inplace method will never
work because the intermediate object on which we are setting values
always behaves as a copy.

For example, when doing 'df[col].method(value, inplace=True)', try
using 'df.method({col: value}, inplace=True)' or df[col] =
df[col].method(value) instead, to perform the operation inplace on the
original object.


  updated_dataset['Purchased'].replace(['No', 'Yes'], inplace=True)
```